



INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA

MANUAL DO PROGRAMADOR

COMPILADORES

Nome: Abel Nkele Canas

Nº Processo: 20231051

Turma: M3

Ano: 3º ano

ÍNDICE

ÍNDICE	2
1. INTRODUÇÃO AO MANUAL	4
2. VISÃO GERAL DA ARQUITETURA	5
3. ORGANIZAÇÃO DO CÓDIGO-FONTE.....	6
4. O PACOTE LEXER — ANALISADOR LÉXICO	6
4.1. Tipo (enum).....	6
4.2. Token (classe)	7
4.3. LexError (exceção)	7
4.4. Lexer (classe principal)	7
5. O PACOTE PARSER — ANALISADOR SINTÁTICO E AST	8
5.1. Metodologia de construção da gramática	8
5.2. A Árvore Sintática Abstrata (AST).....	9
5.3. Preenchimento da Tabela de Símbolos	10
5.4. Alocação recursiva de sub-escopos	10
6. O PACOTE SYMBOL — TABELA DE SÍMBOLOS	11
6.1. SymbolEntry.....	11
6.2. SymbolTable	11
7. O PACOTE SEMANTIC — ANALISADOR SEMÂNTICO	12
7.1. Type — representação de tipos.....	12
7.2. Estrutura da travessia	13
7.3. Mapeamento direto às alíneas do enunciado.....	13
7.4. SemanticError.....	14
7.5. Limitações assumidas (documentadas no próprio código)	14
8. O PACOTE MAIN — PONTO DE ENTRADA.....	15
9. FLUXO DE EXECUÇÃO COMPLETO (PASSO A PASSO)	15
10. COMO COMPILAR O PROJETO	16
Opção A — via NetBeans.....	16
Opção B — via linha de comandos (sem IDE).....	16
11. COMO EXECUTAR O PROJETO	16
12. COMO ESTENDER O COMPILADOR (GUIA PRÁTICO).....	17
12.1. Adicionar uma nova verificação semântica	17

12.2. Adicionar um novo tipo de instrução (statement) à linguagem.....	17
12.3. Adicionar suporte a uma nova classe da biblioteca padrão.....	17
12.4. Depurar um caso em que a Tabela de Símbolos não tem a entrada esperada	17
13. TRATAMENTO E RECUPERAÇÃO DE ERROS	18
13.1. Erros léxicos	18
13.2. Erros sintáticos — modo pânico em três níveis	18
13.3. Erros semânticos	19
14. Convenções de código adotadas	19
15. TESTES E FICHEIROS DE EXEMPLO	20
16. LIMITAÇÕES CONHECIDAS E TRABALHO FUTURO	20
17. REFERÊNCIA RÁPIDA DE CLASSES (API)	21
18. CONCLUSÃO	22
19. Referências Bibliográficas	23

1. INTRODUÇÃO AO MANUAL

Este manual destina-se a quem vai **ler, manter ou estender** o código-fonte do MINI-COMPILADOR. Ele não explica como *usar* o compilador do ponto de vista de quem só quer analisar um ficheiro `.txt/.java` — isso é o papel do *Manual do Utilizador*, em anexo. Aqui o público-alvo é o próprio autor (ou qualquer outro programador) que precise de entender a arquitetura interna, localizar uma classe responsável por um determinado comportamento, ou adicionar uma nova verificação ao analisador semântico.

O projeto está organizado como um **projeto NetBeans (Ant)** dentro da pasta `Compiler/`, com o código-fonte Java em `Compiler/src/com/compiler/`. O compilador implementa as três fases clássicas do *front-end* de um compilador:

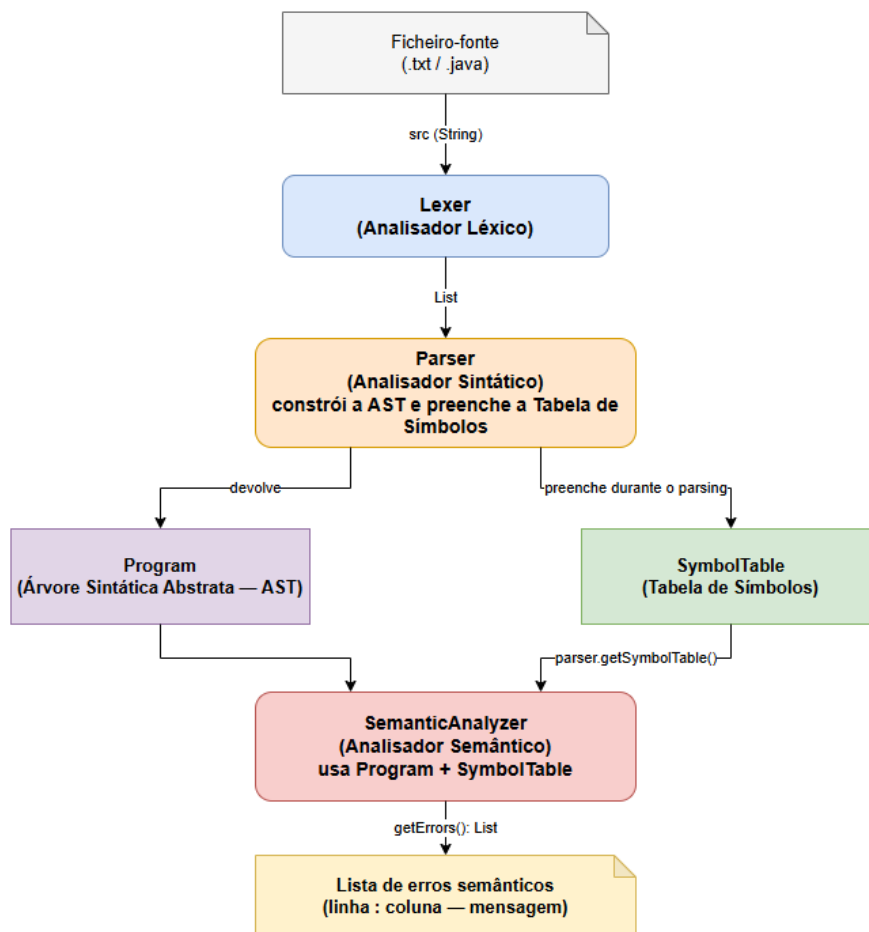
1. **Análise Léxica** (`com.compiler.lexer`) — transforma o texto-fonte numa sequência de *tokens*.
2. **Análise Sintática** (`com.compiler.parser`) — transforma a sequência de tokens numa **Árvore Sintática Abstrata (AST)** e, simultaneamente, preenche a **Tabela de Símbolos** (`com.compiler.symbol`).
3. **Análise Semântica** (`com.compiler.semantic`) — percorre a AST validando as regras semânticas da linguagem, usando a Tabela de Símbolos como fonte de informação sobre cada identificador.

O ponto de entrada de todo o pipeline é a classe `com.compiler.main.Main`.

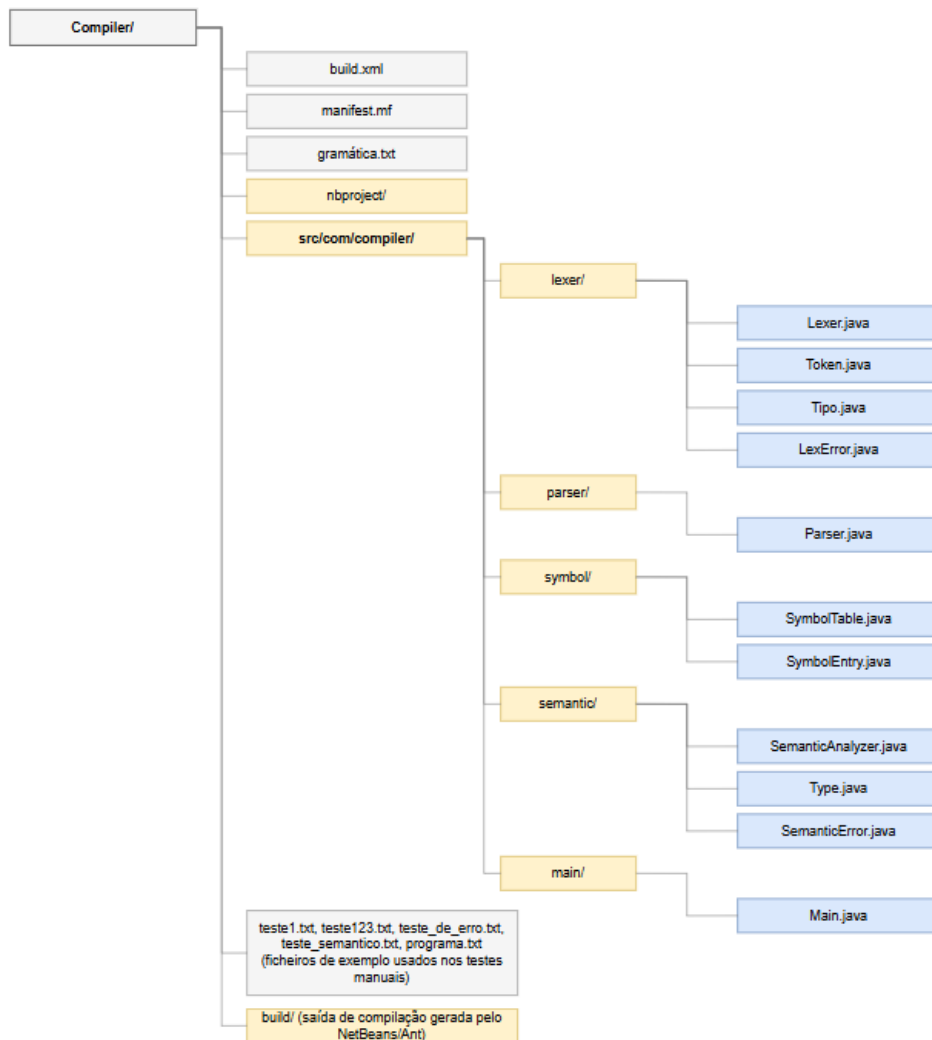
2. VISÃO GERAL DA ARQUITETURA

Um aspeto de desenho importante: **a Tabela de Símbolos é construída pelo `Parser`, não pelo `SemanticAnalyzer`**. O parser já conhece o escopo em que está (classe, método, bloco) no momento em que reconhece uma declaração, por isso é o local mais natural para registar nome, tipo, endereço de memória, tamanho em bytes, escopo e valor inicial de cada símbolo. O SemanticAnalyzer **consome** essa tabela pronta (via `parser.getSymbolTable()`) em vez de a reconstruir, e usa-a para resolver identificadores, validar tipos e detetar declarações duplicadas (a lista de duplicados também é recolhida pelo parser, em `parser.getDuplicateDeclarations()`).

Esta separação de responsabilidades é deliberada e está refletida no Javadoc de SemanticAnalyzer:



3. ORGANIZAÇÃO DO CÓDIGO-FONTE



Não existem dependências externas: todo o compilador usa apenas a biblioteca padrão do Java (`java.util`, `java.nio.file`).

4. O PACOTE LEXER — ANALISADOR LÉXICO

4.1. Tipo (enum)

Define todas as categorias de token reconhecidas: literais (`LITERAL_INTEIRO`, `LITERAL_LONG`, `LITERAL_FLOAT`, `LITERAL_DOUBLE`, `LITERAL_TEXTO`, `LITERAL_CARACTERE`, `LITERAL_ESPECIAL` para `true/false/null`), identificadores, palavras reservadas, todos os operadores (aritméticos, relacionais, lógicos, bit a bit, de atribuição composta, shifts), delimitadores (`(( ) { } [ ] ; , . :)`, `EOF` e `DESCONHECIDO` (para caracteres não reconhecidos)).

4.2. Token (classe)

Estrutura simples e imutável do ponto de vista de uso: `tipo` (um `Tipo`), `lexema` (o texto original), `linha` e `col` (posição no ficheiro-fonte, usada depois pelo parser e pelo analisador semântico para reportar erros com localização exata).

4.3. LexError (exceção)

`RuntimeException` lançada nos três casos em que o scanner não consegue recuperar sozinho: string não fechada, literal de caractere não fechado e comentário de bloco não fechado. Estes casos abortam a análise porque não há um ponto de sincronização seguro dentro do próprio token.

4.4. Lexer (classe principal)

Metodologia: o scanner é implementado como um **Autómato Finito Determinístico (AFD) explícito**, simulado por uma tabela de transições (`transicao(Estado, char): Estado`) sobre um enum de 50+ estados (Q0 a Q63). Cada chamada a `scan()` percorre o código-fonte carácter a carácter, mantendo:

- `estadoAtual` — o estado corrente do autómato;
- `lexemaBuffer` — o lexema acumulado até ao momento;
- `pos`, `linha`, `col` — posição de leitura no ficheiro-fonte.

O ciclo principal (`processarToken`) só faz uma transição de estado quando o próximo carácter é compatível com a categoria de token que já está a ser reconhecida; assim que a transição não é possível (`transicao(...) == null`), o lexema acumulado é fechado (`finalizarToken`) e classificado.

Casos especiais tratados fora da tabela genérica de transições, por simplicidade e clareza:

- **Operadores compostos** (`processarOperadorEspecifico`): em vez de representar todas as combinações de 1 a 4 caracteres (`=`, `==`, `<`, `<=`, `<<`, `<<=`, `>>=`, etc.) apenas na tabela de estados, o lexer resolve diretamente por *lookahead* de 1–3 caracteres a partir do primeiro carácter do operador. Isto simplifica bastante o autómato genérico sem perder o determinismo.
- **Comentários de linha** (`/\`) e **de bloco** (`/\* ... */`, com suporte a aninhamento): tratados por sub-rotinas dedicadas (`pularComentarioLinha`, `pularComentarioBloco`) que **não geram token** — são simplesmente ignorados, avançando `pos/linha/col`.
- **Strings** (`processarString`) e **caracteres** (`processarChar`): tratados por sub-rotinas dedicadas devido à necessidade de reconhecer sequências de escape (`\`, `\\`, `\n`, etc.) sem as interpretar (`\\` seguido de qualquer carácter é aceite literalmente).
- **Números** (`processarNumero`): a tabela de estados distingue inteiro (Q1), sufixo `L/l` (`long`, Q2), ponto decimal (Q3→Q4) e sufixo `F/f/D/d` (Q5), decidindo entre `LITERAL_INTEIRO`, `LITERAL_LONG`, `LITERAL_FLOAT` e `LITERAL_DOUBLE` no fecho do token.

- **Identificadores e palavras reservadas** (`processarIdentificador`): a tabela reconhece sequências de letras/dígitos/_/\$ (estado Q8); no fecho do token, `finalizarToken` consulta as listas `keywordsArray` (50 palavras reservadas de Java) e `literalsArray` (`true`, `false`, `null`) para decidir entre `PALAVRA_RESERVADA`, `LITERAL_ESPECIAL` ou `IDENTIFICADOR`.

Erros léxicos: caracteres não reconhecidos (`Tipo.DESCONHECIDO`) não abortam a análise — são reportados em `System.err` como aviso e ignorados, permitindo que o scanner continue (política de recuperação "descartar e continuar"). Já strings/chars/comentários de bloco não fechados lançam `LexError`, que se propaga até `Main` (não há recuperação parcial dentro de um literal inacabado).

5. O PACOTE `PARSER` — ANALISADOR SINTÁTICO E AST

`Parser.java` é o ficheiro mais extenso do projeto (~2800 linhas) porque concentra três responsabilidades:

1. O **parser descendente recursivo** propriamente dito (reconhecimento da gramática);
2. A **definição da AST** (todas as classes `Node/Statement/Expression` estão aninhadas dentro de `Parser`, como *static nested classes*);
3. O **preenchimento da Tabela de Símbolos** durante o reconhecimento de declarações.

5.1. Metodologia de construção da gramática

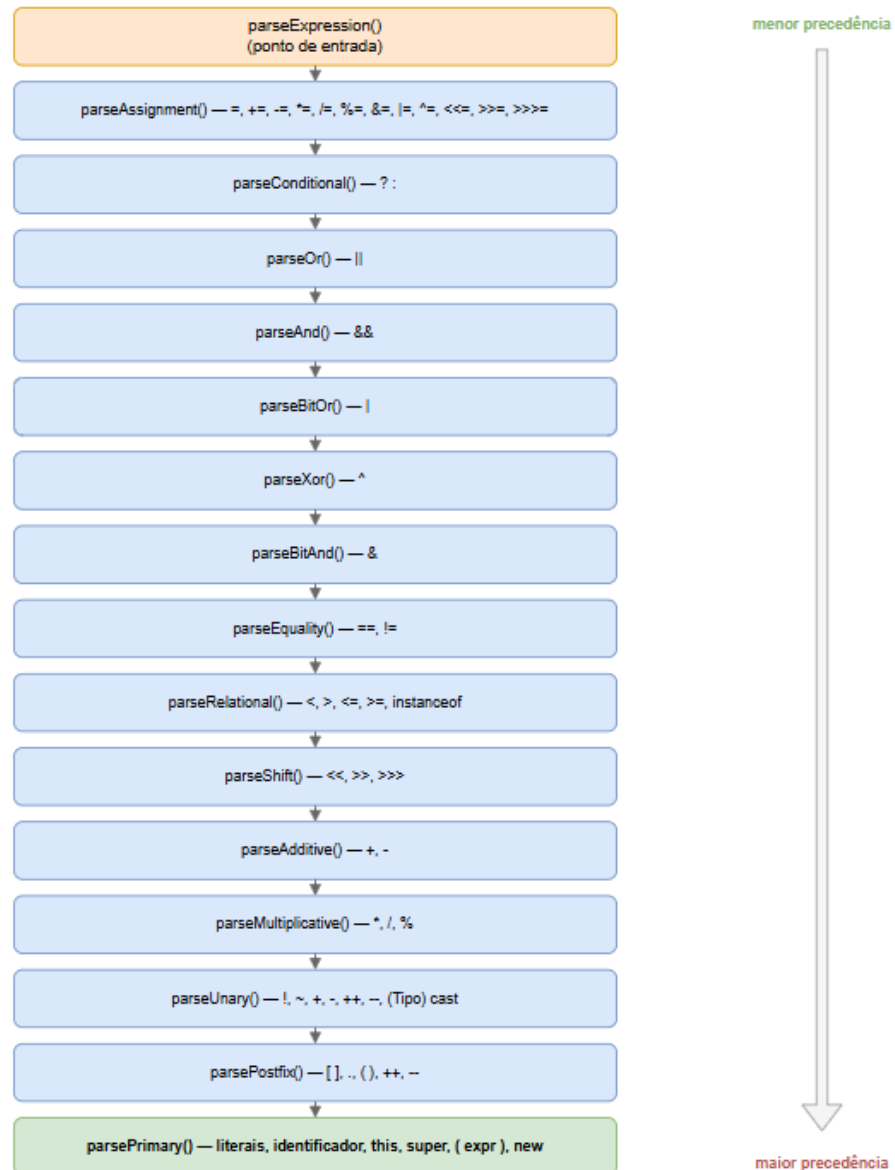
A gramática usada (ver `gramatica.txt` na raiz do projeto `Compiler/`) foi preparada especificamente para permitir um parser **LL(1) / descendente recursivo com lookahead limitado**, eliminando os dois problemas clássicos que impediriam essa técnica:

- **Ambiguidade do "dangling else":** resolvida seguindo a convenção-padrão (o `else` liga-se sempre ao `if` mais próximo sem `else`), através da regra `<instrucaoIf> ::= "if" "(" <expressao> ")" <instrucaoNoElse> ("else" <instrucao>)?`, em que `<instrucaoNoElse>` proíbe explicitamente um `if` "aberto" antes de um `else` do próprio ramo — implementado no código em `parseStatement/parseSelfStatement` fazendo o `else` associar-se sempre à chamada recursiva mais interna.
- **Recursividade à esquerda nas expressões:** a gramática de expressões da linguagem Java é naturalmente recursiva à esquerda (ex.: `expressaoAditiva ::= expressaoAditiva ("+"|"") expressaoMultiplicativa | expressaoMultiplicativa`). Essa recursividade foi eliminada reescrevendo cada nível de precedência como um **laço iterativo** sobre a produção equivalente à direita: `expressaoAditiva ::= expressaoMultiplicativa (("+"|"") expressaoMultiplicativa)*`. Isto é o que se vê em métodos como `parseAdditive()`, `parseMultiplicative()`, `parseShift()`, etc. — todos seguem o padrão "parse do nível abaixo, depois `while` consumindo o operador do próprio nível e o operando seguinte".

Esta reescrita produz uma **hierarquia de precedência** (do menor para o maior), cada um implementado como um método próprio, chamando sempre o nível imediatamente abaixo:

Esta é a mesma ideia por trás de um *Pratt parser*/parser de precedência de operadores, mas implementada de forma totalmente explícita através da hierarquia de métodos acima (sem tabela de precedência em runtime), o que facilita muito a leitura e a depuração.

Cada método chama o nível seguinte e, num laço, consome o(s) operador(es) do seu próprio nível — eliminando a recursividade à esquerda



5.2. A Árvore Sintática Abstrata (AST)

Todas as classes de nó (Program, TypeDeclaration, ClassDeclaration, InterfaceDeclaration, MethodDeclaration, ConstructorDeclaration, FieldDeclaration,

Statement e as suas ~20 subclasses, Expression e as suas ~25 subclasses) estendem a classe base Node, que carrega line/col para permitir localizar qualquer nó no ficheiro-fonte original (usado depois pelo analisador semântico para relatar erros). A associação Token → Node é feita através dos utilitários mark(node, token) e markPos(node, line, col).

Nós de erro dedicados (ErrorStatement, ErrorMember) permitem que o parser **recupere de um erro sintático sem parar completamente**: em vez de abortar, o parser insere um nó de erro no lugar da construção que falhou e continua a processar o resto do ficheiro (ver secção 13).

5.3. Preenchimento da Tabela de Símbolos

Sempre que o parser reconhece uma declaração (campo, parâmetro, variável local, método, construtor, classe, interface), chama addToSymbolTable(name, kind, typeName, arrayDepth, initialValue, modifiers). Este método:

1. Resolve o DataType (primitivo, String, void ou OBJECT com customTypeName) e o respetivo tamanho em bytes (DataType.getSize()).
2. Calcula um **endereço de memória simulado**, através de currentScope.allocateAddress(size) (endereços incrementais a partir de 0x1000, apenas para fins didáticos — não correspondem a memória real).
3. Cria uma SymbolEntry com todos os atributos: nome, tipo, tipo personalizado, tamanho, endereço, caminho do escopo, valor inicial, modificadores, dimensão de array, linha e coluna.
4. Tenta inserir no currentScope (SymbolTable.addSymbol); se já existir um símbolo com o mesmo nome nesse escopo, a inserção falha e a duplicata é registada em duplicateDeclarations — **não é lançado um erro sintático**; a decisão de como reportar isso é da responsabilidade do analisador semântico (verificação b, secção 7).

Cada vez que o parser entra num novo escopo (corpo de classe, de método, de construtor, de bloco { }), cria uma nova SymbolTable filha (new SymbolTable(currentScope, nomeDoEscopo)) e restaura o escopo anterior ao sair (padrão save/restore visível em visitMethodDeclaration, parseBlock, etc. — o mesmo padrão é usado no SemanticAnalyzer para currentClassName/currentClassScope/currentReturnType).

5.4. Alocação recursiva de sub-escopos

Note-se que um escopo filho herda o próximo endereço livre do escopo pai (this.nextMemoryAddress = parent.nextMemoryAddress no construtor de SymbolTable) — isto é uma simplificação didática que evita colisões de endereço entre variáveis de escopos diferentes, sem modelar de forma precisa uma pilha de ativação real.

6. O PACOTE SYMBOL — TABELA DE SÍMBOLOS

6.1. SymbolEntry

Guarda exatamente os atributos exigidos pelo enunciado para cada identificador:

Atributo	Campo Java	Descrição
Nome	name	Identificador declarado
Tipo de dado	dataType / customTypeName	Primitivo (DataType) ou tipo definido pelo utilizador
Endereço de memória	memoryAddress	Endereço simbólico atribuído pelo parser
Tamanho em bytes	sizeInBytes	Deriva de <code>DataType.getSize()</code> (ex.: <code>int=4</code> , <code>double=8</code> , <code>referência=8</code>)
Escopo	scope	Caminho completo do escopo (Classe.metodo)
Valor de atribuição	initialValue	Valor/expressão do inicializador, se existir
Dimensão	arrayDimensions	Número de <code>[]</code> do tipo
Categoria (kind)	kind	CLASS, INTERFACE, FIELD, STATIC_FIELD, PARAMETER, LOCAL_VARIABLE, METHOD, CONSTRUCTOR
Modificadores	modifiers	public, static, final, etc.
Posição	line, column	Localização da declaração

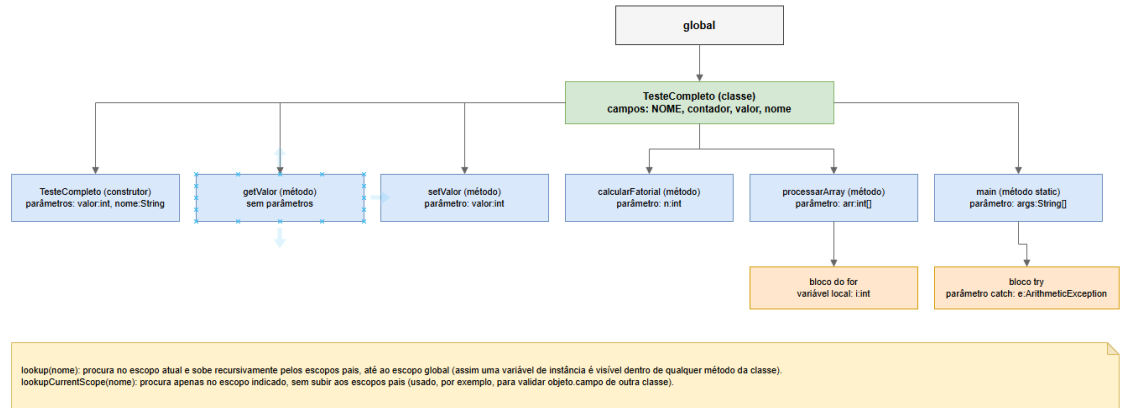
Também expõe utilitários: `getFullTypeName()` (nome do tipo com sufixos `[]`), `isPrimitive()/isReference()` e `getMemoryRegion()` (mapeamento didático para "Stack", "Heap" ou "Área de Classe", consoante o kind).

6.2. SymbolTable

Implementa uma tabela de símbolos **hierárquica** (uma árvore de escopos): cada `SymbolTable` tem um parent opcional e uma lista de children. As operações principais:

- `addSymbol(entry)` — insere no escopo atual; devolve `false` em caso de colisão de nome (usado para detetar duplicados).
- `lookup(name)` — procura no escopo atual e, recursivamente, nos escopos ascendentes (parent) até ao escopo global — é assim que uma variável declarada numa classe é visível dentro dos seus métodos, e que uma variável de um bloco externo é visível num bloco aninhado.

- `lookupCurrentScope(name)` — procura **apenas** no escopo atual, sem subir aos pais; usado para validar membros de uma classe específica (ex.: acesso `objeto.campo`) sem "vazar" para o escopo de quem está a chamar.
- `printFullTable()` — imprime a tabela de símbolos completa e formatada (usada por `Main` para o relatório de compilação)



•

7. O PACOTE SEMANTIC — ANALISADOR SEMÂNTICO

Esta é a componente central deste exame. A classe `SemanticAnalyzer` recebe, no construtor, a `Program` (raiz da AST) e a lista de `Parser.DuplicateDeclaration` recolhida pelo parser, e expõe um único método público de análise: `analyze()`.

7.1. Type — representação de tipos

Antes de descrever as verificações, é essencial entender `Type`, a classe que o analisador semântico usa para representar o **tipo de qualquer expressão** calculado durante a travessia da AST. Cada `Type` guarda:

- `base` (um `SymbolEntry.DataType`: `INT`, `DOUBLE`, `BOOLEAN`, `STRING`, `OBJECT`, `VOID`, ...);
- `className` (para tipos referência definidos pelo utilizador);
- `arrayDepth` (número de dimensões de array);
- `classRef` (marca um `Type` que representa uma referência a um *nome de classe*, como em `System.out`, e não um valor);
- flags internas `nullType` e `error`.

Os métodos mais importantes:

- `isAssignableTo(Type target)` — implementa as regras de **compatibilidade de atribuição** (usadas nas verificações c, d e e): promoção numérica (*widening*) `byte`→`short`/char→`int`→`long`→`float`→`double`, aceitação de `null` para qualquer tipo referência, `Object` como super-tipo universal de qualquer referência, e igualdade estrita de `className/arrayDepth` para arrays e classes definidas pelo utilizador.

- `numericResult(Type a, Type b)` — determina o tipo resultante de uma operação aritmética binária, seguindo a promoção padrão do Java (se um dos operandos é `double`, o resultado é `double`; senão `float`; senão `long`; senão `int`).
- Predicados de conveniência: `isNumeric()`, `isBoolean()`, `isIntegral()`, `isString()`, `isArrayType()`, `isReference()`, `isVoid()`, `isError()`, `isNull()`.
- `Type.ERROR` — um tipo sentinela usado para **propagar erros sem os reportar em duplicado**: qualquer operação envolvendo `Type.ERROR` é considerada automaticamente compatível (`isAssignableTo` devolve `true`), evitando uma "cascata" de erros semânticos derivados de um único erro original (ex.: usar uma variável não declarada numa expressão maior não deve gerar 3 ou 4 erros adicionais).

7.2. Estrutura da travessia

`analyze()` segue esta sequência:

1. `collectClassScopes(...)` — pré-processamento que regista, para cada classe/interface (incluindo aninhadas), o mapeamento nome da classe → `SymbolTable` do seu corpo. Isto é necessário porque o acesso a membros de **outra** classe (`objeto.metodo()`, `Classe.campoEstatico`) só pode ser resolvido depois de todas as classes do programa serem conhecidas — independentemente da ordem em que aparecem no ficheiro-fonte.
2. `reportDuplicateDeclarations()` — converte cada `Parser.DuplicateDeclaration` recolhida durante o parsing num `SemanticError` (verificação **b**).
3. Para cada `TypeDeclaration` do programa, `visitTypeDeclaration` percorre recursivamente todos os seus membros (`visitClassMember`), instruções (`visitStatement`) e expressões (`visitExpression`), acumulando `Type` como valor de retorno de `visitExpression` — este é o clássico padrão de **verificação de tipos por atribuição de tipo sintetizado** (o tipo de um nó é calculado a partir dos tipos dos seus filhos).

Durante a travessia, três variáveis de instância mantêm o "contexto atual" (à semelhança do que o Parser faz com `currentScope`):

- `currentClassName` / `currentClassScope` — para resolver `this`, chamadas de método sem qualificador e campos herdados do escopo da própria classe;
- `currentReturnType` — para validar `return` (verificação relacionada a tipos de retorno, incluída na alínea c/e do enunciado).

7.3. Mapeamento direto às alíneas do enunciado

Alínea do enunciado	Onde está implementado	Mensagem de erro típica
a) Variável não declarada	<code>visitIdentifier</code> — <code>scope.lookup(name)</code> devolve <code>null</code> e o nome não é uma classe conhecida	Variável 'x' não declarada
b) Variável declarada duas vezes no mesmo	<code>reportDuplicateDeclarations()</code> , a partir de	Símbolo 'x' já declarado no escopo 'Y'

escopo	SymbolTable.addSymbol que devolveu false no parser	
c) Incompatibilidade de tipos de dados	visitBinary / incompatibleOperands (operadores aritméticos, relacionais, lógicos, bit a bit) e visitUnary/visitPostfix	Operador '+' não pode ser aplicado a tipos incompatíveis 'boolean' e 'int'
d) Tipos dos argumentos de funções/métodos	checkArguments, chamado a partir de visitMethodInvocation e visitNew (construtores)	Argumento 2 de 'soma' tem tipo 'String', esperado 'int'
e) Atribuição de tipo incompatível a outra variável	checkAssignable (declarações com inicializador) e visitAssignment (operador = e atribuições compostas +=, -=, ...)	Não é possível atribuir um valor do tipo 'String' para a variável 'z' do tipo 'int'
f) Argumentos das estruturas de controlo	checkBooleanCondition (if/while/do-while/for/assert), visitForEach (compatibilidade do tipo do elemento com a variável do for-each) e visitSwitch (tipo da expressão e dos case)	Condição de 'if' deve ser do tipo boolean, mas é 'int'

Além destas seis verificações mínimas exigidas, o analisador implementado cobre também: tipo de retorno de métodos (visitReturn), compatibilidade de tipos no operador ternário (visitConditional), existência de campos/métodos antes de aceder a eles (visitFieldAccess, visitMethodInvocation), correspondência de número de argumentos em chamadas e construções new (checkArguments, visitNew), tipo do índice de array (visitArrayAccess), tipo das dimensões de new T[n] (visitNewArray) e compatibilidade de tipo entre o elemento de um array e a variável de um for-each (visitForEach).

7.4. SemanticError

Estrutura mínima — line, col, message — com um método format() que produz a string final "Erro semântico em L:C - mensagem". Todas as mensagens de erro são geradas em português para consistência com o resto da ferramenta.

7.5. Limitações assumidas (documentadas no próprio código)

O Javadoc de SemanticAnalyzer documenta explicitamente três simplificações conscientes, coerentes com o âmbito de um "mini-compilador":

1. **Não há resolução de sobrecarga (overloading)** de métodos/construtores: o parser já trata um segundo método/construtor com o mesmo nome, no mesmo escopo, como um símbolo duplicado (alínea b). Este é um efeito colateral aceite:

a tabela de símbolos deste mini-compilador modela "um nome → um símbolo por escopo", tal como pedido no enunciado, e não "um nome → uma lista de assinaturas".

2. **Não há modelagem de herança** entre classes definidas pelo utilizador — apenas `Object` é tratado como super-tipo universal de qualquer tipo referência (`Type.isAssignableTo`).
3. **Classes da biblioteca padrão** (`System`, `Math`, `String`, coleções, etc.) são aceites através de uma *whitelist* fixa (`KNOWN_BUILTIN_CLASSES`), porque este compilador não modela `imports` nem o *classpath* do JDK — o objetivo é validar o código do utilizador, não recompilar a biblioteca padrão de Java.

8. O PACOTE MAIN — PONTO DE ENTRADA

`Main.main(String[] args)`:

1. Lê o caminho do ficheiro-fonte do primeiro argumento da linha de comandos, ou usa `"teste_de_erro.txt"` por omissão.
2. Corre o `Lexer` (`lexer.scan()`) para obter a lista de tokens que o `Parser` vai consumir — mas **não imprime** o código-fonte nem a lista de tokens por omissão (ver nota abaixo).
3. Cria o `Parser` com a lista de tokens e chama `parseProgram()`.
4. Se o parser reportou erros sintáticos, imprime-os, mas **continua** a análise na mesma (o parser já recuperou usando modo pânico — ver secção 13), porque nem sempre um erro sintático local invalida o resto do ficheiro.
5. Imprime a tabela de símbolos completa (`symbolTable.printFullTable()`).
6. Cria o `SemanticAnalyzer` com a `Program` e as duplicatas, chama `analyze()`, e imprime a lista de erros semânticos (se existirem).
7. Produz um resumo final colorido (verde/vermelho via códigos ANSI) com o número total de erros (sintáticos + semânticos).

Nota de desenho: `Main.java` mantém, comentados (não removidos), os blocos que reimprimiam um banner decorativo, o código-fonte lido e a lista completa de tokens do `Lexer`. Esses blocos foram desativados porque não fazem parte do que o **enunciado do Analisador Semântico** pede para ser demonstrado: o enunciado exige que o semântico valide a AST com base nos atributos já gravados na Tabela de Símbolos (endereço, tipo, dimensão, valor, escopo, tamanho em bytes) — não que o programa reimprima o código-fonte ou o fluxo de tokens do `Lexer` (essa é uma preocupação da fase léxica, já coberta noutro momento de avaliação). Ficam apenas comentados, e não apagados, para que continuem disponíveis caso seja necessário depurar o `Lexer` isoladamente — bastando descomentá-los.

9. FLUXO DE EXECUÇÃO COMPLETO (PASSO A PASSO)

1. `Main.main(args)`
2. `src = Files.readString(caminho)`
3. `tokens = new Lexer(src).scan()`
4. `parser = new Parser(tokens)`

```
5. program = parser.parseProgram() // preenche symbolTable ao longo do
   caminho
6. symbolTable = parser.getSymbolTable()
7. duplicates = parser.getDuplicateDeclarations()
8. analyzer = new SemanticAnalyzer(program, duplicates)
9. analyzer.analyze()
10. erros = parser.getErrors() + analyzer.getErrors()
11. imprime tokens, tabela de símbolos, erros sintáticos, erros semânticos, resumo
    final
```

10. COMO COMPILAR O PROJETO

Opção A — via NetBeans

Abrir a pasta `Compiler/` como projeto NetBeans (já contém `nbproject/`) e usar **Run** → **Build Project** (ou Shift+F11). O `build.xml` (Ant) trata da compilação e da geração do `.jar` com o Main-Class correto.

Opção B — via linha de comandos (sem IDE)

A partir da pasta `Compiler/`:

```
# compilar todas as classes para uma pasta de saída
find src -name "*.java" > sources.txt
javac -d build/classes -encoding UTF-8 @sources.txt
```

No Windows/PowerShell, equivalente:

```
Get-ChildItem -Recurse -Filter *.java src | ForEach-Object { $_.FullName } >
sources.txt
javac -d build\classes -encoding UTF-8 "@sources.txt"
```

Requisitos: **JDK 17 ou superior** (o código usa `switch` com expressão de padrão (`switch (kind) { case X -> ...}`), *records* implícitos via `Optional`, e sintaxe de *pattern matching* `instanceof` como em `if (td instanceof ClassDeclaration cd)`).

11. COMO EXECUTAR O PROJETO

Depois de compilado (classes em `build/classes` ou `Compiler/build/classes` se usado o NetBeans):

```
cd build/classes
java com.compiler.main.Main caminho/para/o/ficheiro.txt
```

Se nenhum argumento for passado, o programa tenta abrir `teste_de_erro.txt` no diretório de trabalho atual.

Nota sobre acentuação no terminal Windows: como as mensagens de erro estão em português com acentos, é recomendável correr com a *code page* UTF-8 e forçar a codificação da JVM, para evitar caracteres corrompidos na consola:


```
chcp 65001
java "-Dfile.encoding=UTF-8" "-Dstdout.encoding=UTF-8" com.compiler.main.Main
ficheiro.txt
```

(Ver mais detalhes no *Manual do Utilizador*, secção "Execução".)

12. COMO ESTENDER O COMPILADOR (GUIA PRÁTICO)

Esta secção existe para acelerar tarefas comuns de manutenção.

12.1. Adicionar uma nova verificação semântica

1. Identificar o(s) nó(s) da AST relevante(s) em `Parser.java` (ex.: uma nova instrução ou uma nova forma de expressão).
2. Se ainda não existir, adicionar o caso correspondente em `SemanticAnalyzer.visitStatement` ou `visitExpression`.
3. Implementar a regra usando os predicados já existentes em `Type` (`isNumeric()`, `isAssignableTo(...)`, etc.) — evitar comparar `DataType` diretamente para preservar a lógica de *widening* já centralizada em `Type`.
4. Reportar o erro com `errors.add(new SemanticError(line, col, mensagem))`, reutilizando `positionOf(expr)` para obter a posição mesmo quando a expressão está entre parênteses.
5. Devolver sempre `Type.ERROR` (ou um tipo coerente) do método de verificação, para não propagar erros em cascata.

12.2. Adicionar um novo tipo de instrução (statement) à linguagem

1. Adicionar a regra na gramática (`gramática.txt`).
2. Criar a classe do nó AST como *nested class* de `Parser`, estendendo `Statement`.
3. Implementar o método `parseXYZStatement()` no `Parser`, e ligar a chamada em `parseStatement()` (o método central de despacho por *lookahead* da palavra-chave).
4. Adicionar o caso correspondente em `SemanticAnalyzer.visitStatement`.

12.3. Adicionar suporte a uma nova classe da biblioteca padrão

Adicionar o nome à constante `KNOWN_BUILTIN_CLASSES` em `SemanticAnalyzer.java`. Isto é suficiente para que identificadores como `NomeDaClasse.metodo(...)` deixem de ser reportados como "variável não declarada" — o analisador não valida os membros dessas classes (não faz parte do âmbito do mini-compilador), apenas aceita a referência ao nome.

12.4. Depurar um caso em que a Tabela de Símbolos não tem a entrada esperada

Verificar sempre, pela ordem:

1. Se a declaração foi de facto processada por `addToSymbolTable` no `Parser` (colocar um `System.out.println` temporário, ou breakpoint).

2. Se o escopo (`currentScope`) no momento da declaração é o esperado — erros deste tipo aparecem tipicamente quando se esquece de restaurar `currentScope` ao sair de um bloco/método (padrão *save/restore*).
3. Se a procura está a usar `lookup` (sobe aos escopos pai) quando devia usar `lookupCurrentScope` (não sobe), ou vice-versa.

13. TRATAMENTO E RECUPERAÇÃO DE ERROS

13.1. Erros léxicos

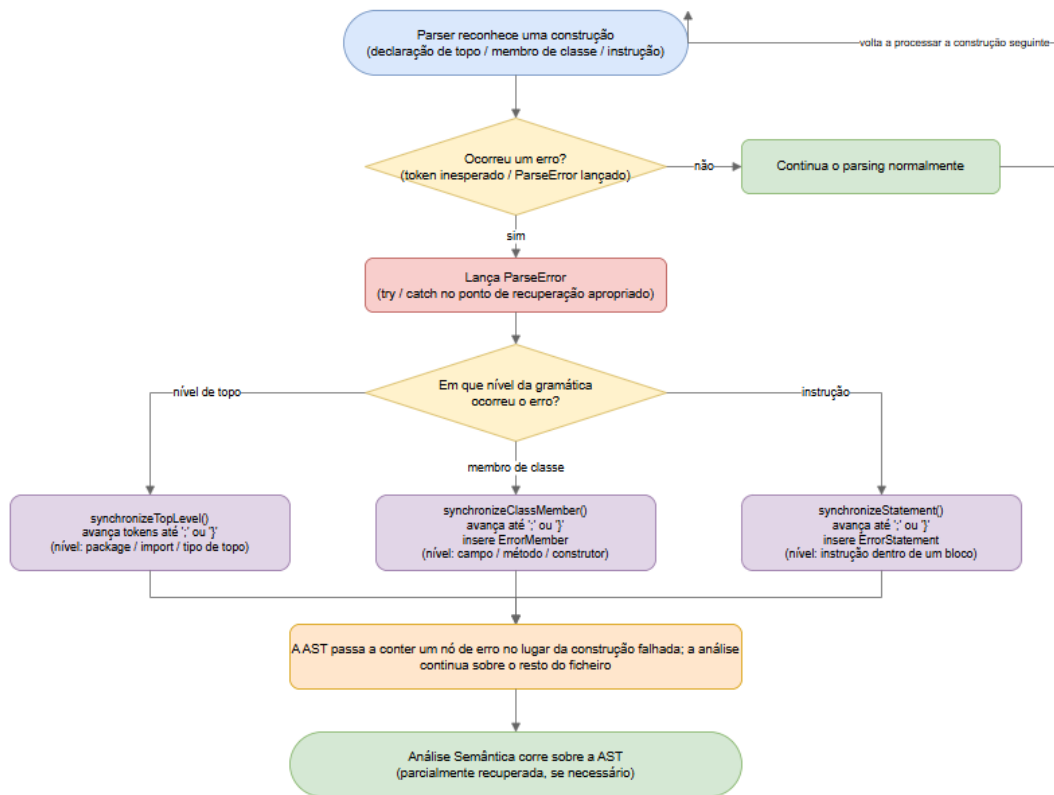
Ver secção 4.4: caracteres desconhecidos são ignorados com aviso; strings/chars/comentários de bloco não fechados lançam `LexError` (erro fatal, sem recuperação).

13.2. Erros sintáticos — modo pânico em três níveis

O `Parser` implementa **recuperação de erros em modo pânico** (*panic-mode error recovery*), com três pontos de sincronização independentes, cada um usado no nível da gramática apropriado:

- `synchronizeTopLevel()` — usado quando falha o parsing de uma declaração de `package/import`/tipo de topo; avança tokens até ao próximo `;` ou `}`.
- `synchronizeClassMember()` — usado quando falha o parsing de um membro de classe (campo, método, construtor); permite que o resto da classe continue a ser analisado mesmo que um membro esteja malformado.
- `synchronizeStatement()` — usado quando falha o parsing de uma instrução dentro de um bloco; o erro é isolado à instrução atual.

Cada ponto de recuperação está envolvido num `try { ... } catch (ParseError error) { synchronizeXyz(); }`, e o nó em falta é substituído por um nó de erro (`ErrorStatement/ErrorMember`) para que a AST permaneça estruturalmente válida e a análise semântica possa continuar sobre o resto do ficheiro.



Mensagens de erro sintático especializadas (ex. `errorMissingSemicolon`, `errorMissingOpenBrace`, `errorMissingExpression`) produzem texto mais específico do que um genérico "token inesperado", indicando o contexto (ex.: "esperado ';' após declaração de variável").

13.3. Erros semânticos

O `SemanticAnalyzer` **nunca lança exceções** durante a análise normal — todos os problemas são acumulados na lista `errors` e devolvidos no fim via `getErrors()`. Isto permite reportar **todos** os erros semânticos de um ficheiro numa só execução, em vez de parar no primeiro. O uso sistemático de `Type.ERROR` como valor de retorno para expressões inválidas evita cascatas de erros derivados do mesmo problema original.

14. Convenções de código adotadas

- Identificadores de domínio (classes, métodos, variáveis do próprio compilador) e mensagens de erro visíveis ao utilizador estão em **português**, alinhado com o facto de este ser um projeto académico em português.
- Cada fase expõe uma API mínima e consistente: `hasErrors()` + `getErrors(): List<String>`.

- Padrão *save/restore* de contexto (`currentScope`, `currentClassName`, `currentReturnType`) sempre que se entra/sai de um escopo — nunca é feita mutação de contexto sem o correspondente restauro no mesmo método.
- Nós da AST são classes aninhadas estáticas de `Parser`, agrupadas por categoria (`Declaration`, `Statement`, `Expression`) através de uma hierarquia de classes base simples, sem *frameworks* de terceiros.

15. TESTES E FICHEIROS DE EXEMPLO

A raiz de `Compiler/` inclui vários ficheiros de entrada usados como testes manuais de regressão (não há *testes automatizados* com JUnit neste projeto — a verificação é feita correndo o `Main` sobre cada ficheiro e inspecionando a saída):

Ficheiro	Propósito
<code>programa.txt</code>	Programa simples e correto (tipos primitivos, <code>if</code> , comentários)
<code>teste1.txt</code>	Cobre variações de <code>if/else/ternário</code>
<code>teste123.txt</code>	Programa mais completo: construtores, campos estáticos, arrays, <code>try/catch/finally</code> , recursão
<code>teste_de_erro.txt</code>	Ficheiro propositadamente malformado (testa recuperação sintática)
<code>teste_semantico.txt</code>	Ficheiro com as seis classes de erro semântico (a–f) lado a lado, para validar o analisador semântico

Recomenda-se, ao alterar o `SemanticAnalyzer`, correr sempre o compilador sobre `teste_semantico.txt` e confirmar que as sete linhas de erro esperadas continuam a ser reportadas (ver o *Manual do Utilizador*, secção de exemplos, para o conteúdo exato e a saída esperada).

16. LIMITAÇÕES CONHECIDAS E TRABALHO FUTURO

- Sem sobrecarga de métodos/construtores (ver 7.5).
- Sem modelagem de herança entre classes do utilizador.
- Sem verificação de exceções verificadas (`throws`) nem de fluxo de controlo (ex.: "variável pode não ter sido inicializada", "falta um `return` em todos os caminhos").
- Sem geração de código (o âmbito deste exame termina na análise semântica — não há back-end).
- A tabela de símbolos usa endereços de memória simbólicos incrementais, não uma modelação real de pilha/heap.

Estas limitações são intencionais e proporcionais ao âmbito de um "mini-compilador" académico, sendo aliás as mesmas assumidas e documentadas no próprio Javadoc de `SemanticAnalyzer`.

17. REFERÊNCIA RÁPIDA DE CLASSES (API)

Classe	Pacote	Responsabilidade principal
Lexer	lexer	Converte código-fonte em List<Token>
Token	lexer	Um token: tipo, lexema, linha, coluna
Tipo	lexer	Enum de categorias de token
LexError	lexer	Exceção para erros léxicos fatais
Parser	parser	Parser descendente recursivo; define a AST; preenche a SymbolTable
Parser.Program	parser	Raiz da AST
Parser.DuplicateDeclaration	parser	Registo de um símbolo duplicado detetado durante o parsing
SymbolTable	symbol	Tabela de símbolos hierárquica (uma por escopo)
SymbolEntry	symbol	Atributos de um símbolo (nome, tipo, endereço, escopo, ...)
SemanticAnalyzer	semantic	Percorre a AST e valida as regras semânticas (a-f + extras)
Type	semantic	Representação de tipo usada nas verificações semânticas
SemanticError	semantic	Um erro semântico: linha, coluna, mensagem
Main	main	Ponto de entrada; orquestra as três fases e imprime o relatório

18. CONCLUSÃO

Este manual reuniu toda a informação necessária para que um programador — o próprio autor, num momento futuro, ou qualquer outra pessoa que herde este código — consiga situar-se rapidamente na arquitetura do MINI-COMPILADOR, localizar a classe responsável por um determinado comportamento e alterar ou estender o sistema com segurança. A organização em três pacotes bem delimitados (`lexer`, `parser` + `symbol`, `semantic`), a separação clara de responsabilidades entre a construção da Tabela de Símbolos (feita durante o parsing) e a sua utilização (feita na análise semântica), e a documentação explícita das limitações assumidas são, em conjunto, o que torna este projeto fácil de manter apesar da sua dimensão. Fica também demonstrado que é possível construir as três fases clássicas de um *front-end* de compilador — léxica, sintática e semântica — usando apenas as funcionalidades da biblioteca padrão de Java, sem qualquer dependência externa, o que facilita tanto a compreensão do código como a sua avaliação académica.

19. Referências Bibliográficas

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2ª edição. Boston: Pearson/Addison Wesley, 2006.

LOUDEN, Kenneth C. *Compiler Construction: Principles and Practice*. Boston: PWS Publishing Company, 1997.

APPEL, Andrew W. *Modern Compiler Implementation in Java*. 2ª edição. Cambridge: Cambridge University Press, 2002.

GOSLING, James; JOY, Bill; STEELE, Guy; BRACHA, Gilad; BUCKLEY, Alex; SMITH, Daniel; BIERMAN, Gavin. *The Java Language Specification*, edição correspondente ao Java SE 17. Oracle America, Inc.

PARR, Terence. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Raleigh: The Pragmatic Bookshelf, 2010.